

Использование языка программирования Julia для решения математических задач

Часть I

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

Глава 4 Функции в Julia

4.1 Базовый синтаксис определения функции.

Структура функции в языке Julia:

```
function name(список параметров)
тело функции
return результат # не обязателен, если return не указан, функция
возвращает результат последнего вычисленного выражения.
end
```

Ключевое слово `return` обеспечивает выход из функции и может встречаться в функции несколько раз.

```
function test(n)
    if n < 0
        return "n < 0"
    elseif n == 0
        return "n == 0"
    else
        return "n > 0"
    end
end
```

Вызов функции:

```
name(список параметров)
```

Рассмотрим на примере функции нахождения корней квадратного уравнения.

```
In [34]: function quadratic_roots(a, b, c)
          d = b^2 - 4*a*c
          if d < 0
              return "Нет действительных корней"
          else
              x1 = (-b + sqrt(d)) / (2a)
              x2 = (-b - sqrt(d)) / (2a)
              return x1, x2
          end
      end

      roots = quadratic_roots(1, -3, 2)
```

Out[34]: (2.0, 1.0)

4.2 Однострочные функции.

В языке программирования Julia можно определять функции не только с использованием традиционного синтаксиса **function ... end**, но и с помощью более компактного синтаксиса, который позволяет определить функцию в одну строку. Такие функции называются **однострочными функциями**.

`function_name(список параметров) = выражение`

```
In [35]: add(x, y) = x + y
          println(add(5, 6))
```

11

4.3 Анонимные функции.

Анонимные функции в Julia – это функции, которые не имеют имени и определяются прямо в месте их использования.

Анонимные функции в Julia обычно создаются с использованием стрелочного синтаксиса `->`, который указывает на определение функции. Стрелка разделяет параметры функции от тела функции.

```
In [36]: r = x -> x^2
          println(r(5))
```

25

Анонимные функции могут принимать несколько аргументов, и это делается аналогично обычным функциям.

```
In [37]: r = (x, y) -> x + y
          println(r(3, 4))
```

Анонимные функции могут возвращать несколько значений, если они заключены в кортеж (или массив). Это полезно, если нужно вернуть сразу несколько результатов.

```
In [40]: r = (x, y) -> (x + y, x * y)
println(r(5, 6))
```

```
(11, 30)
```

По умолчанию анонимные функции в Julia предполагают, что тело функции состоит из одного выражения. Однако можно использовать более сложные блоки кода, если они обернуты в **begin ... end**. В таком случае анонимная функция может содержать несколько выражений.

```
In [41]: r = x -> begin
           z = x^2
           z + 1
         end
println(r(3))
```

```
10
```

```
In [42]: arr = [1, 2, 3, 4, 5]
r = map(x -> x^2, arr) # map применяет анонимную функцию ко всем элементам массива
println(r)
```

```
[1, 4, 9, 16, 25]
```

4.4 Блоки do

Блоки **do** создают анонимную функцию и передают ее в качестве первого аргумента внешней функции при вызове.

```
In [43]: map([1, 2, 3]) do x
           2*x
         end
```

```
Out[43]: 3-element Vector{Int64}:
          2
          4
          6
```

```
In [56]: map([1, 2, 3], [1, 2, 3]) do x, y
           println("$x + $y = ", x + y)
           println("2 * $x = ", 2*x)
         end
```

```
1 + 1 = 2
2 * 1 = 2
2 + 2 = 4
2 * 2 = 4
3 + 3 = 6
2 * 3 = 6
```

```
Out[56]: 3-element Vector{Nothing}:
 nothing
 nothing
 nothing
```